

Agentic AI for Tech Stack



Agentic AI Orchestration and Application Layer



Engage and Think



An AI assistant reviews user interviews, product feedback, and competitor websites.
At first, the summaries and feature ideas seem useful.

Over time, teams notice issues:

- Suggestions are no longer relevant to current goals
- Some responses feel generic or outdated.
- The system cannot explain how it forms conclusions.

What might be missing from how this system is set up?

Learning Objectives

By the end of this lesson, you will be able to:

- 🔗 Explain why orchestration is critical for enabling multi-step autonomous workflows in agentic AI systems
- 🔗 Analyze agentic frameworks such as LangGraph, AutoGen, CrewAI, and LangChain to select appropriate architectures for building AI agents
- 🔗 Examine how routing enhances task success, efficiency, and user satisfaction
- 🔗 Apply routing logic to choose between structured and dynamic approaches for different use cases
- 🔗 Evaluate trade-offs between natural conversation (chat) and structured input (form-fill) for specific scenarios

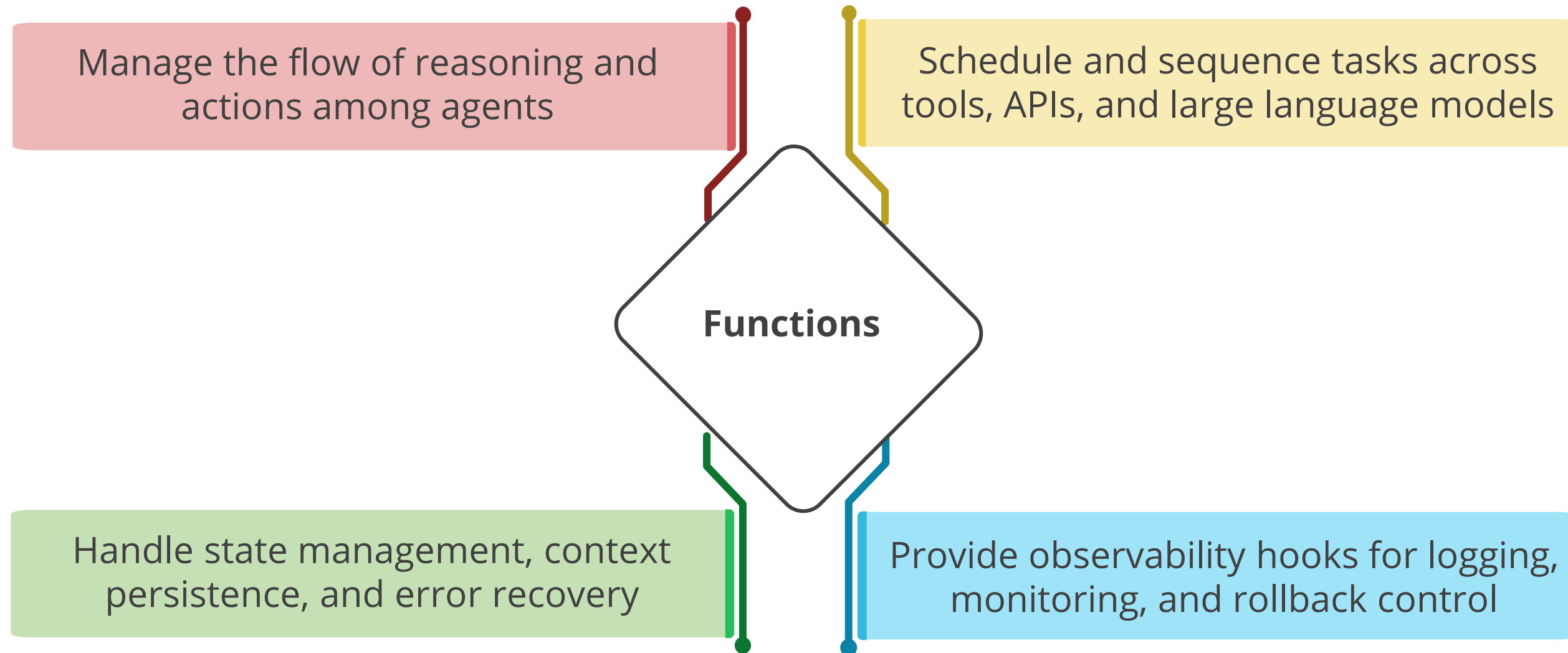




Foundations of the Orchestration Layer

Introduction to Orchestration Layer

This layer serves as the central control system in the agentic AI tech stack. It acts as middleware and performs the following functions:



Agentic Frameworks in the Orchestration Layer

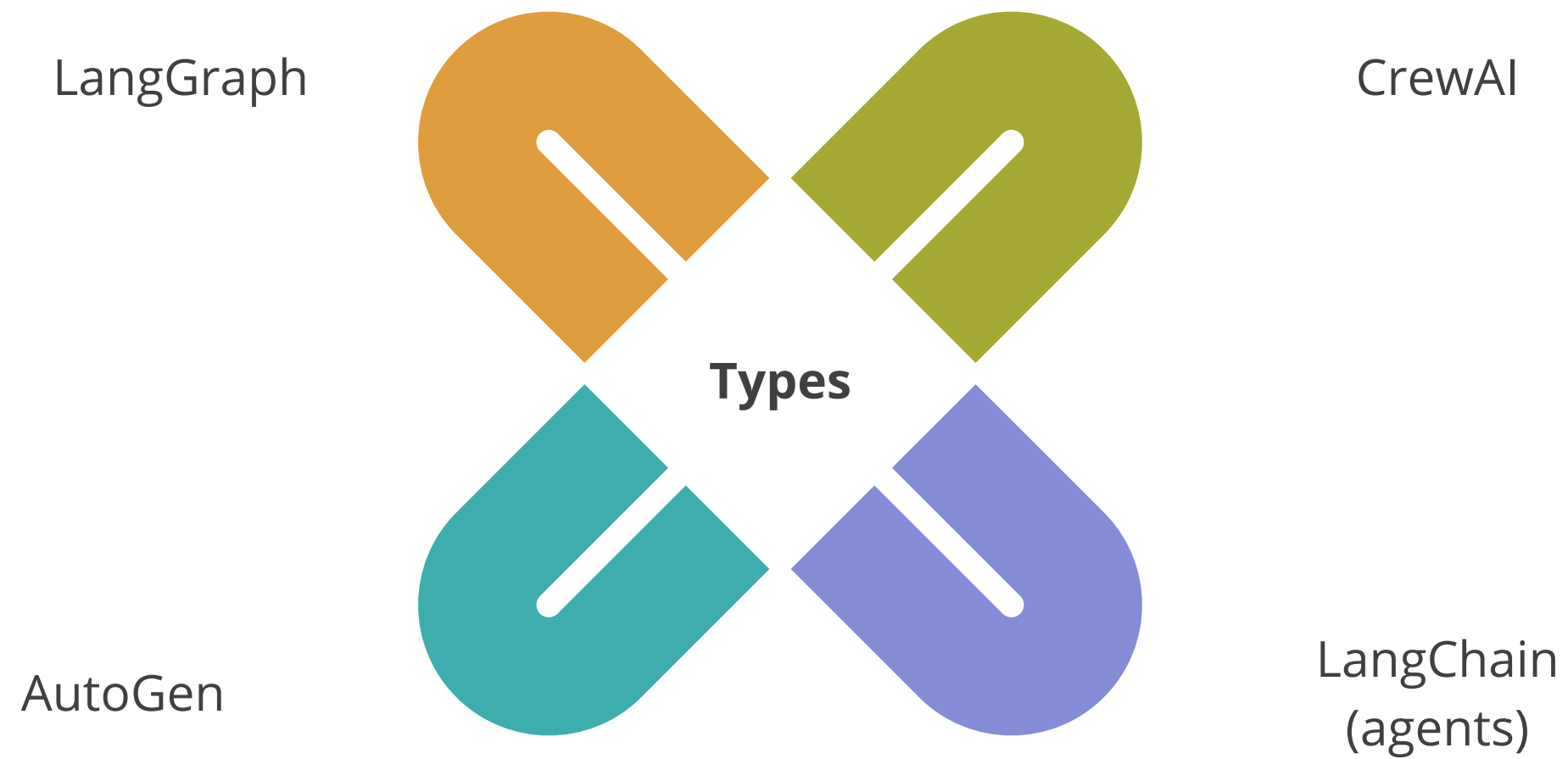
An agentic framework is a structured system that enables autonomous or semi-autonomous agents to perform tasks, make decisions, and collaborate effectively with other agents or humans.

They offer the following benefits:



Types of Agentic Orchestration Frameworks

These frameworks support multi-agent workflows for agentic AI systems.



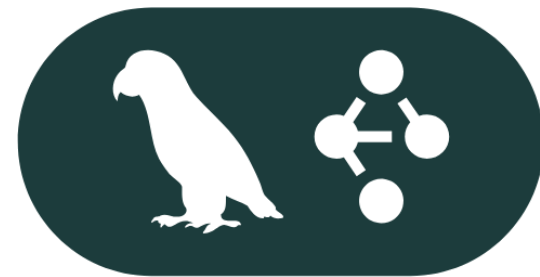
Agentic Orchestration Frameworks: Key Differences

The following are the key differences between LangGraph, CrewAI, AutoGen, and LangChain (agents):

Feature/ Aspect	LangGraph	CrewAI	AutoGen	LangChain (agents)
Primary purpose	Framework for building stateful, multi-step LLM workflows as directed graphs	Framework for multi-agent collaboration with roles and workflows	Framework for multi-agent conversations and tool-augmented LLMs	Library for LLM-powered agents that use tools and memory
Core concept	Nodes = tools/LLMs/functions, Edges = transitions; flow managed by graph execution	Agents work together as a <i>crew</i> with predefined goals and step-by-step delegation	Agents communicate with each other or humans, orchestrating problem-solving	Single or multiple agents that dynamically choose tools to solve tasks
Architecture	Graph-based execution engine; supports conditional branching, loops, memory persistence	Role-based agents (for example, researcher, writer) with defined responsibilities	Conversation-based orchestration of multiple agents with reasoning loops	Planner-executor or ReAct-style agents that reason and act in steps

Introduction to LangGraph

It is an open-source framework for building graph-based AI workflows.

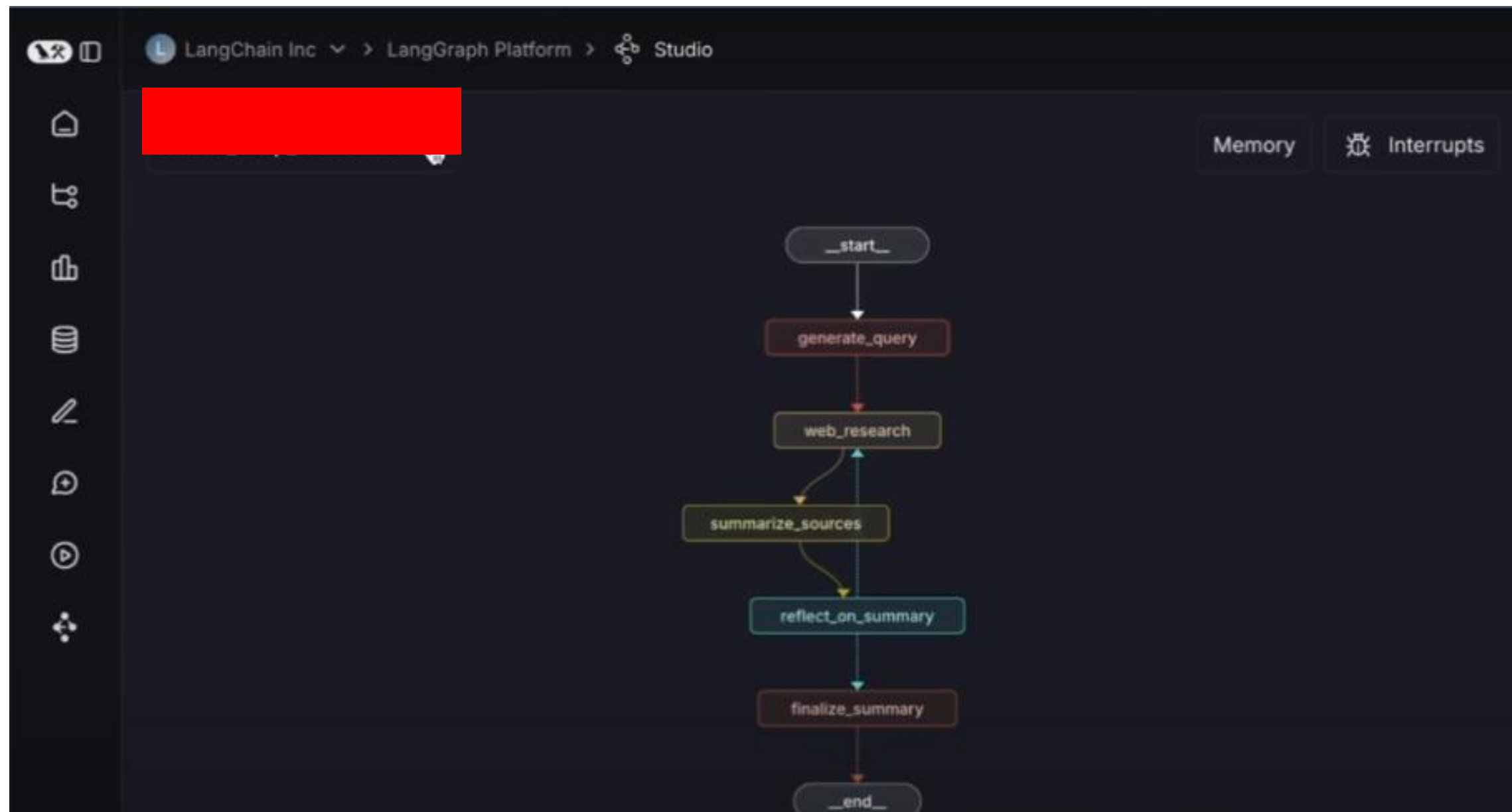


LangGraph

It enables structured and dynamic decision-making by modeling workflows as directed graphs, where nodes define actions and edges control execution flow.

Introduction to LangGraph

The following image shows the LangGraph Platform Studio interface, specifically a visual workflow diagram for a user-defined project.



LangGraph: Workflows and Agents

LangGraph uses workflows and agents to coordinate actions and manage complex tasks.

Workflow

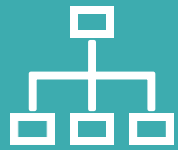
It is a sequence of tasks that are executed to achieve specific goals.

Agents

These are autonomous units that execute tasks in workflows and interact with external systems.

Agents make decisions, while workflows define the order and structure for executing those decisions.

LangGraph: Key Features



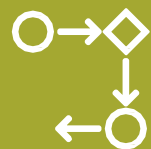
Graph-based execution: Structures tasks as a graph, enabling dynamic routing



Multi-agent support: Enables multiple AI agents to collaborate and coordinate decisions efficiently



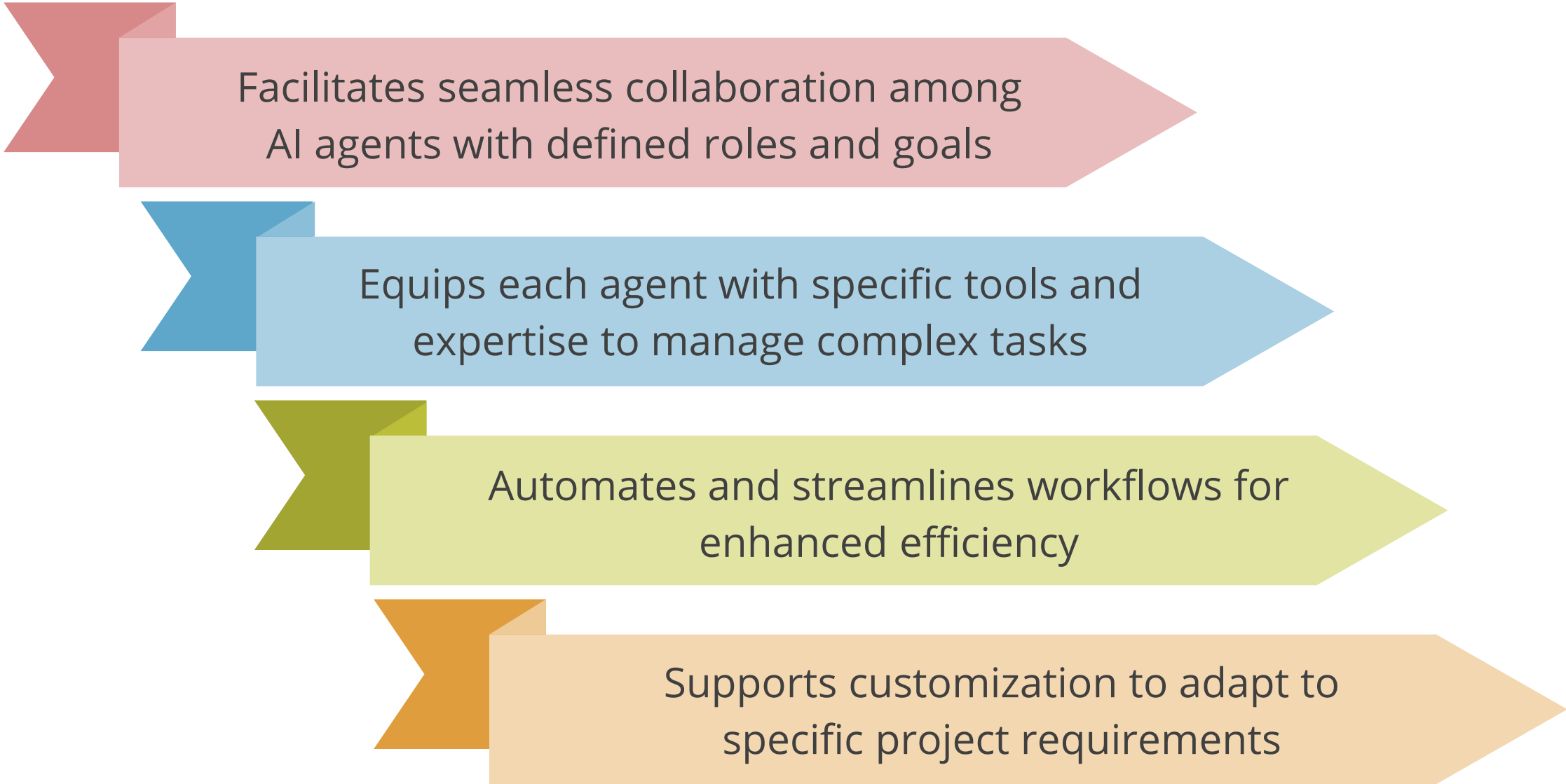
LangChain integration: Enhances LLM-based workflows through seamless compatibility



Flexible execution paths: Nodes and edges dynamically adjust the workflow based on input conditions

Introduction to CrewAI

It is an advanced framework for building and managing teams of autonomous AI agents. Its key capabilities include:



Facilitates seamless collaboration among AI agents with defined roles and goals

Equips each agent with specific tools and expertise to manage complex tasks

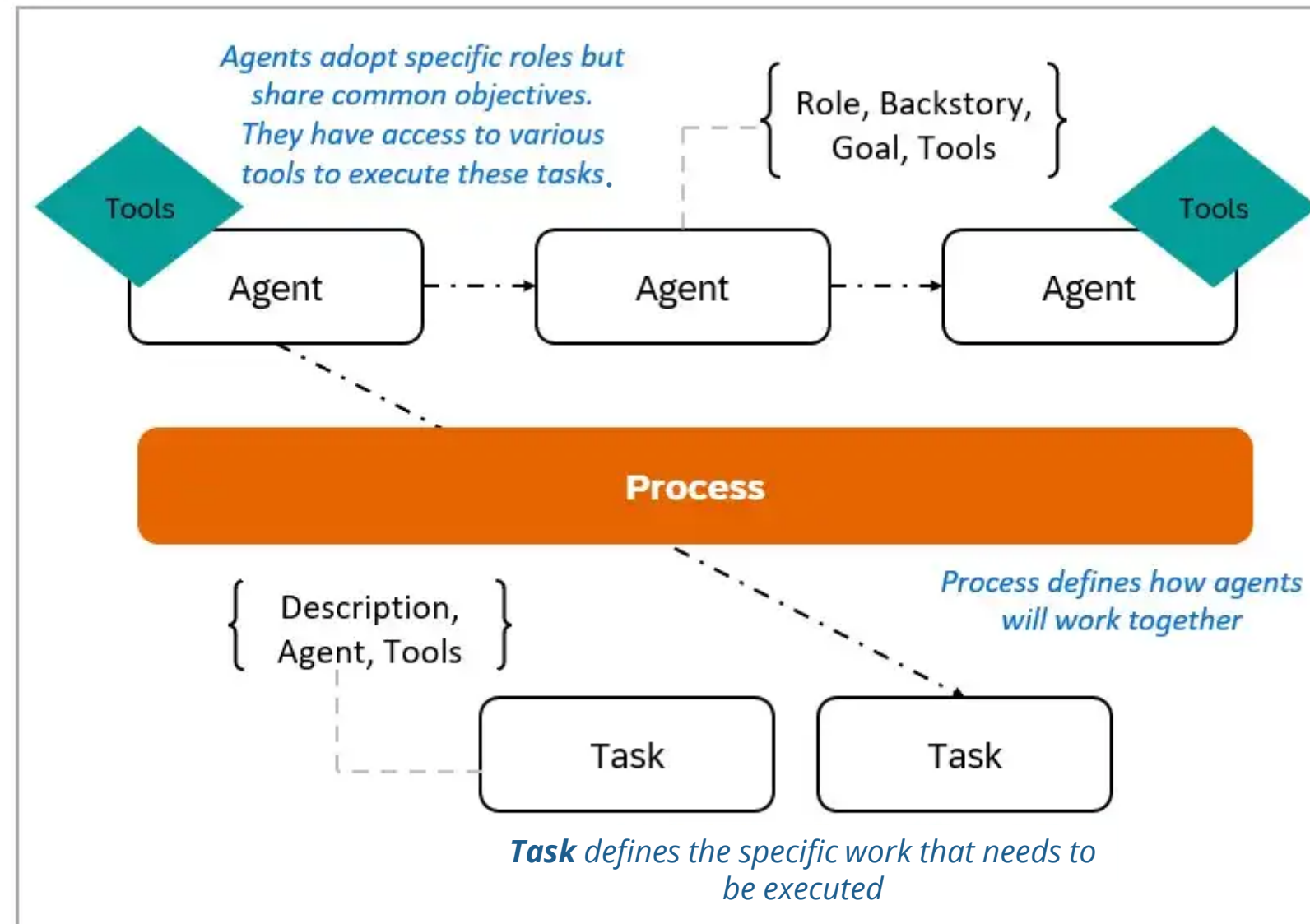
Automates and streamlines workflows for enhanced efficiency

Supports customization to adapt to specific project requirements

CrewAI: Working

It solves problems using a combination of agents, tools, tasks, and defined processes.

Crew



Introduction to AutoGen

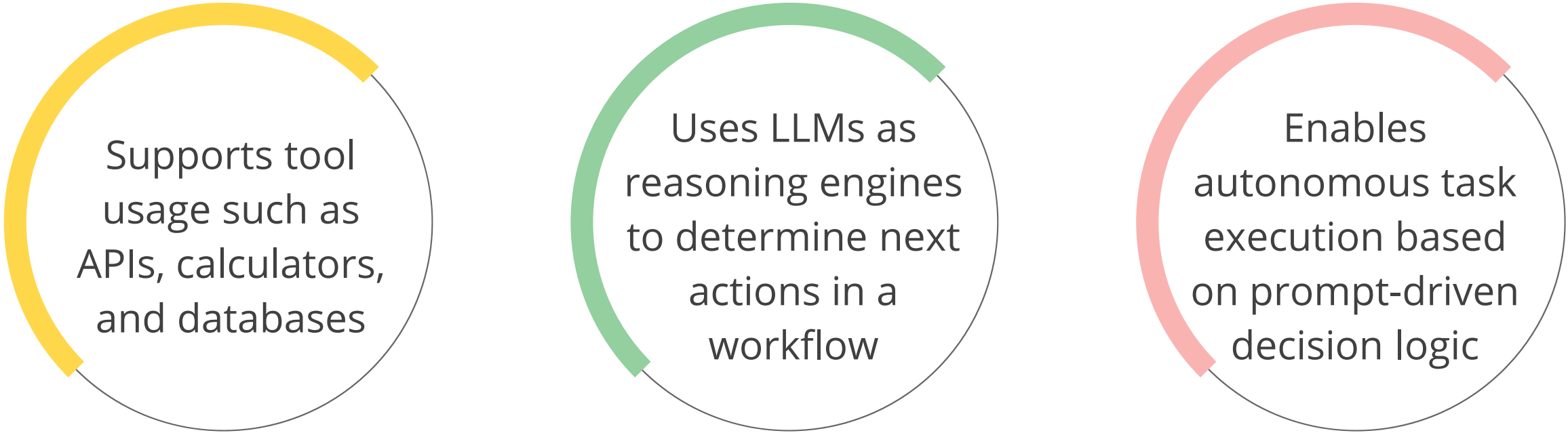
It is an open-source framework from Microsoft that streamlines the creation of conversational multi-agent applications by offering ready-made tools, components, and guidelines, eliminating the need for developers to build everything from scratch.

It focuses on LLM-to-LLM conversations.

It supports both autonomous and human-in-the-loop agents.

What Is LangChain?

It helps developers build applications that use large language models (LLMs) to reason, plan, and interact with external tools. Its key features include:



Supports tool usage such as APIs, calculators, and databases

Uses LLMs as reasoning engines to determine next actions in a workflow

Enables autonomous task execution based on prompt-driven decision logic

Planning, tool use, and memory support enhance the agent's overall capabilities.

Planning, Tool Calling, and Memory Support

In agentic AI systems, the three core capabilities of planning, tool calling, and memory support enable autonomous reasoning, adaptive decision-making, and contextual interaction.

Planning

It refers to the AI agent's ability to break down a high-level goal into smaller tasks, determine their sequence, and adapt dynamically based on intermediate results.

Tool calling

It is the agent's ability to invoke external APIs, databases, or custom functions to autonomously gather data or perform actions.

Memory support

It enables agents to retain context across multiple steps or interactions, making reasoning more coherent and personalized.

To support these capabilities, agentic AI systems must also intelligently route queries to the right tools.

Tool Routing Based on Query Type

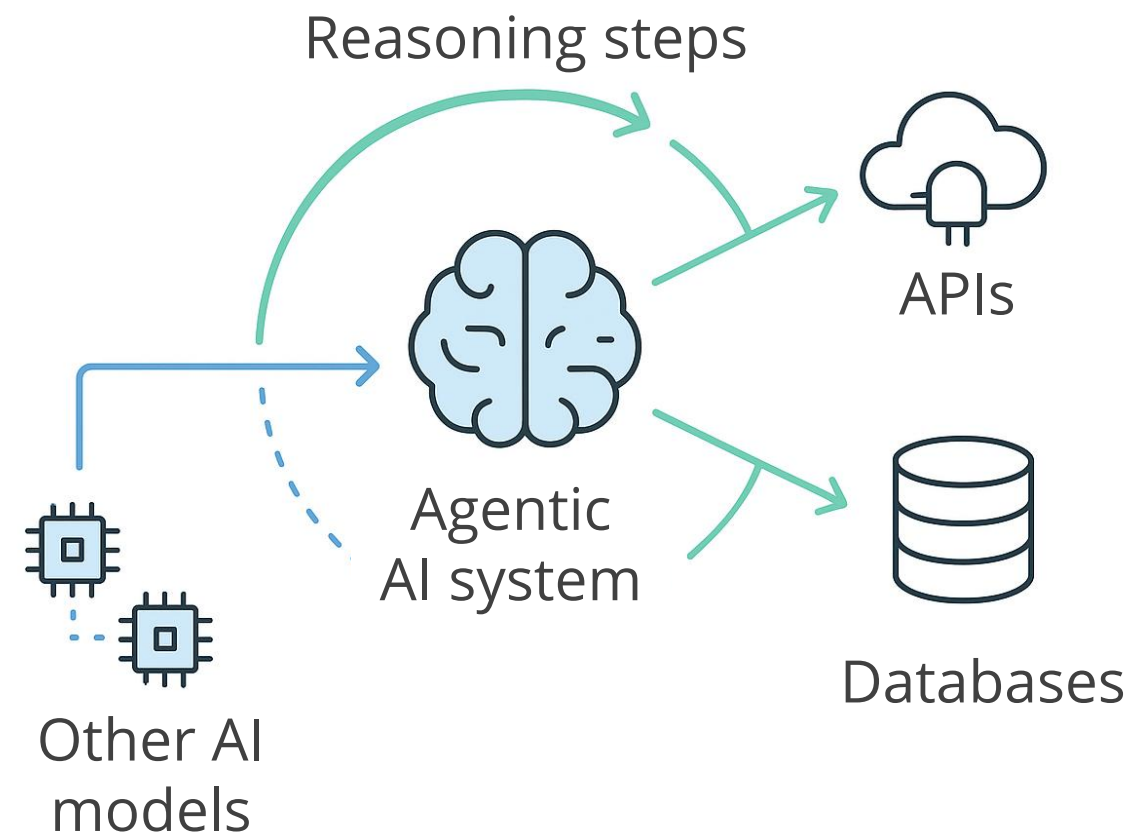
In agentic AI systems, agents must classify queries, understand user intent, and route to the appropriate tools autonomously.

Consider the following questions when designing a routing mechanism:

- **Routing logic:** Which tool or sequence of tools should be used?
- **Classification:** What type of query or task is being asked?
- **Execution:** How should the result be executed and returned?

Errors from Misused Tool Calls in Agentic AI

Agentic AI systems rely on tool calls such as APIs, databases, or model-to-model interfaces. These support multi-step reasoning and external system integration.



Routing Tool Mechanism

Agents use routing tools to classify queries, select the right tools, and generate accurate responses in three steps.

1. Query classification

The agent uses natural language understanding (NLU) or embedding similarity to classify the query type (Example: Informational, analytical, transactional).

2. Tool selection

Based on the classification, the agent selects the most relevant tools, via routing chains in LangChain, graph nodes in LangGraph, or multi-agent delegation in CrewAI.

3. Execution and response

The tool processes the query, while the agent can be designed to perform additional checks before responding.

Reflection and Replanning Loops in Agentic AI

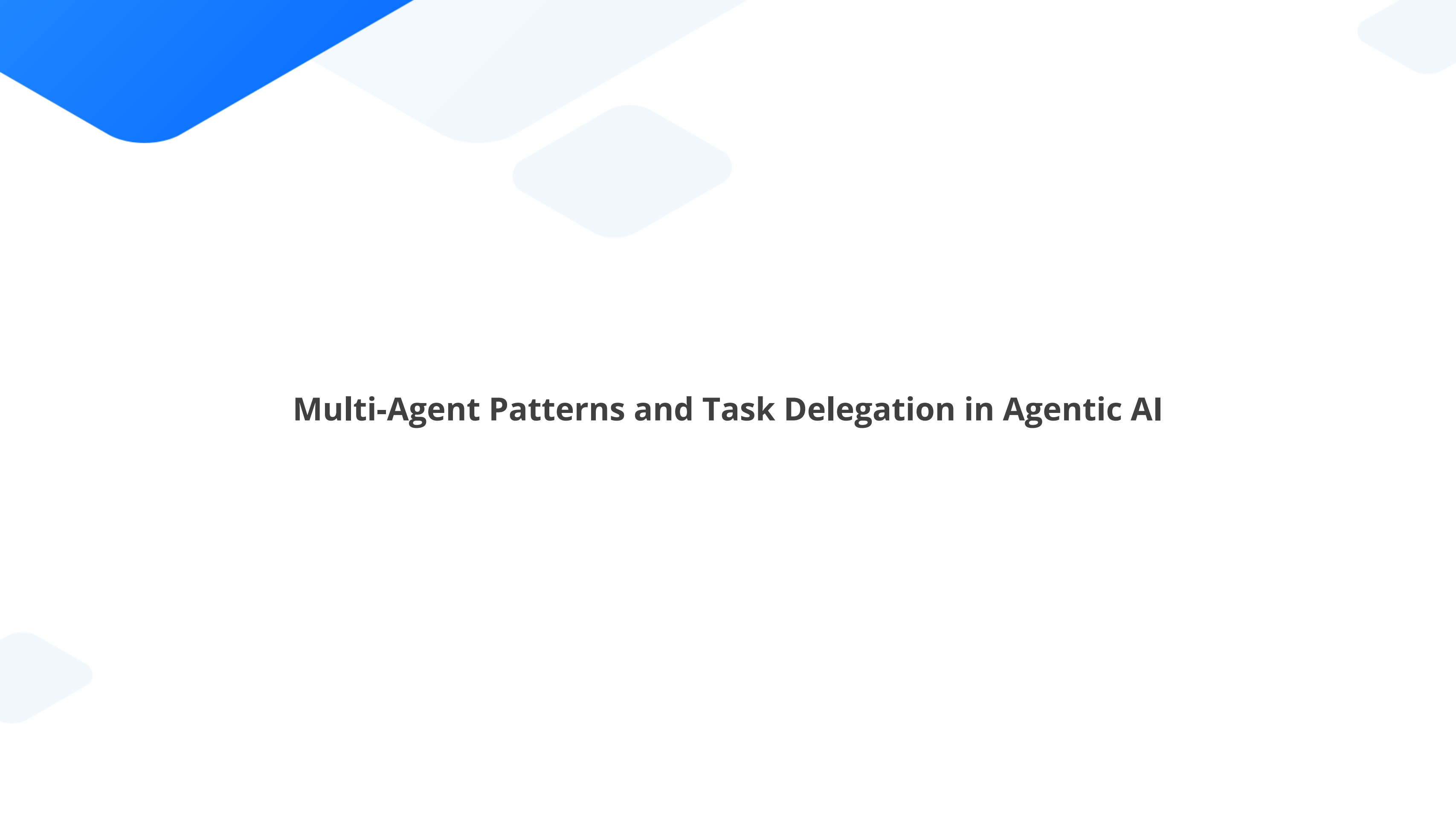
These loops let agents self-evaluate, learn from mistakes, and adjust plans dynamically.

Reflection

- The agent reviews output quality and correctness before proceeding.
- It uses self-critique or reviewer agents to identify errors, inconsistencies, or missed objectives.

Replanning

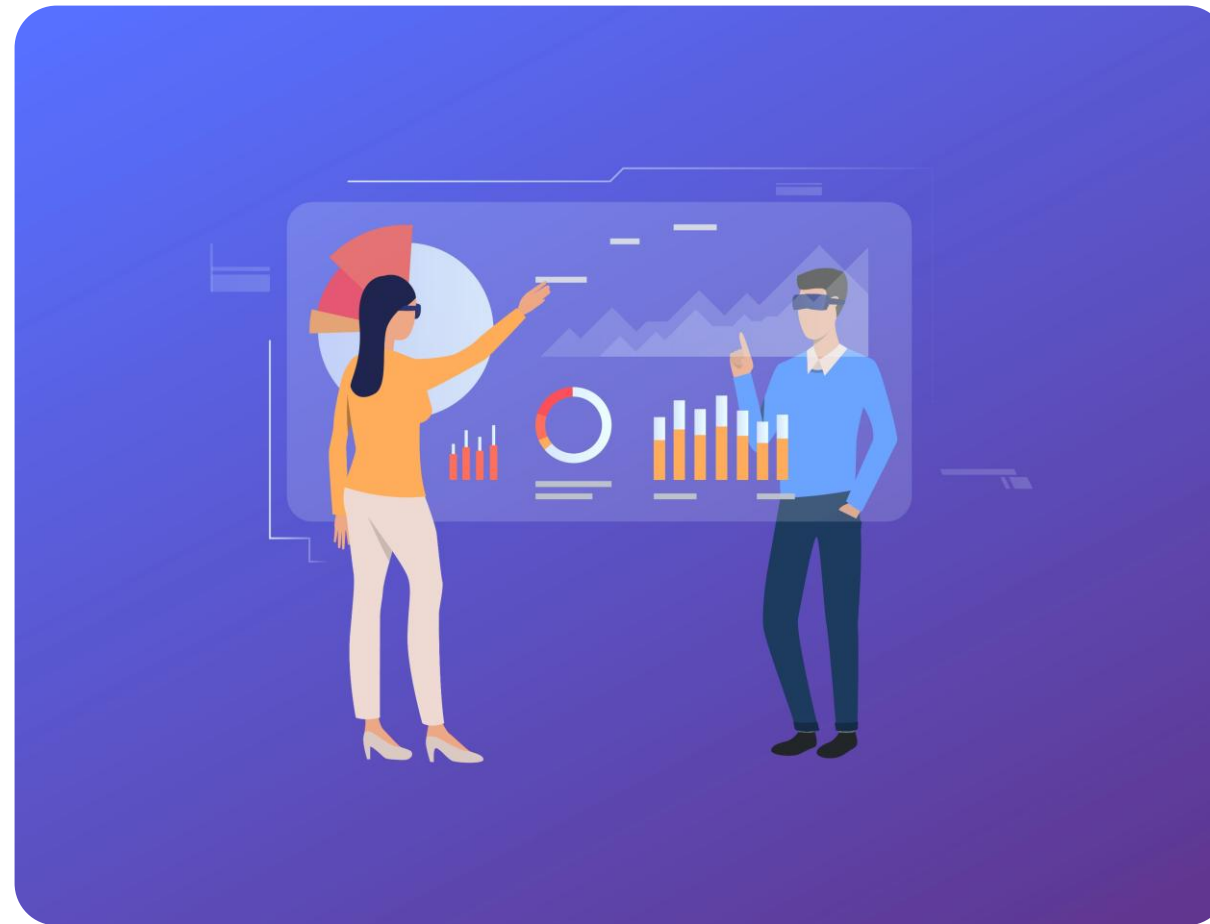
- After reflection or changing conditions, the agent updates its plan or creates a new one.
- This happens if actions fail, new information emerges, or priorities shift.



Multi-Agent Patterns and Task Delegation in Agentic AI

Overview of Multi-Agent Patterns

Multi-agent patterns provide structured ways to organize and coordinate specialized AI agents, each with a defined role to collaboratively achieve complex goals.



These patterns simulate cross-functional collaboration, similar to how product managers delegate tasks to marketing, design, engineering, and research teams.

Multi-Agent Patterns

These patterns simulate cross-functional collaboration among specialized agents, just like PMs delegate tasks to teams such as marketing, design, and engineering.

Pattern	Description	PM analogy
Manager-Worker	A manager agent breaks down tasks and assigns them to <i>worker</i> agents.	PM delegating tasks to marketing, design, and engineering teams
Planner-executor-reviewer	A planner creates a plan, executor's complete tasks, and a reviewer evaluates outcomes.	PM planning product roadmap, teams executing, and QA/review team validating deliverables
Specialized agent teams	Agents grouped by expertise (for example, research, analytics, UX) collaborate and exchange information.	Cross-functional squads working on a single product feature
Debate or critique pattern	Multiple agents argue or critique solutions to reach the best decision.	Product brainstorming sessions or prioritization meetings

Task Delegation in Agentic AI

This refers to the ability of an AI agent, or orchestrator agent, to assign specific subtasks to other specialized agents based on their capabilities. For instance:



Chain-of-Thought (CoT)

It is a prompting technique that guides a language model to solve problems step by step by making its reasoning process explicit.

Key benefits of the chain-of-thought technique are:

Breaks down
complex tasks into
smaller intermediate
steps

Enhances model
accuracy by reducing
reasoning shortcuts

Enables
transparency in
decision-making for
better control

Plan-and-Execute Agents

Plan-and-execute agents split tasks into planning and execution phases, using tools or reasoning in each step.

Separate the planning phase from the execution phase

Use one module (planner) to generate a sequence of actions or goals

Use another module (executor) to carry out each action in order

Improve efficiency, scalability, and reasoning in multi-step workflows

Chain-of-Thought (CoT) vs. Plan-and-Execute Agents in Agentic AI

The following highlights key differences between CoT and plan-and-execute agents:

(CoT) agents

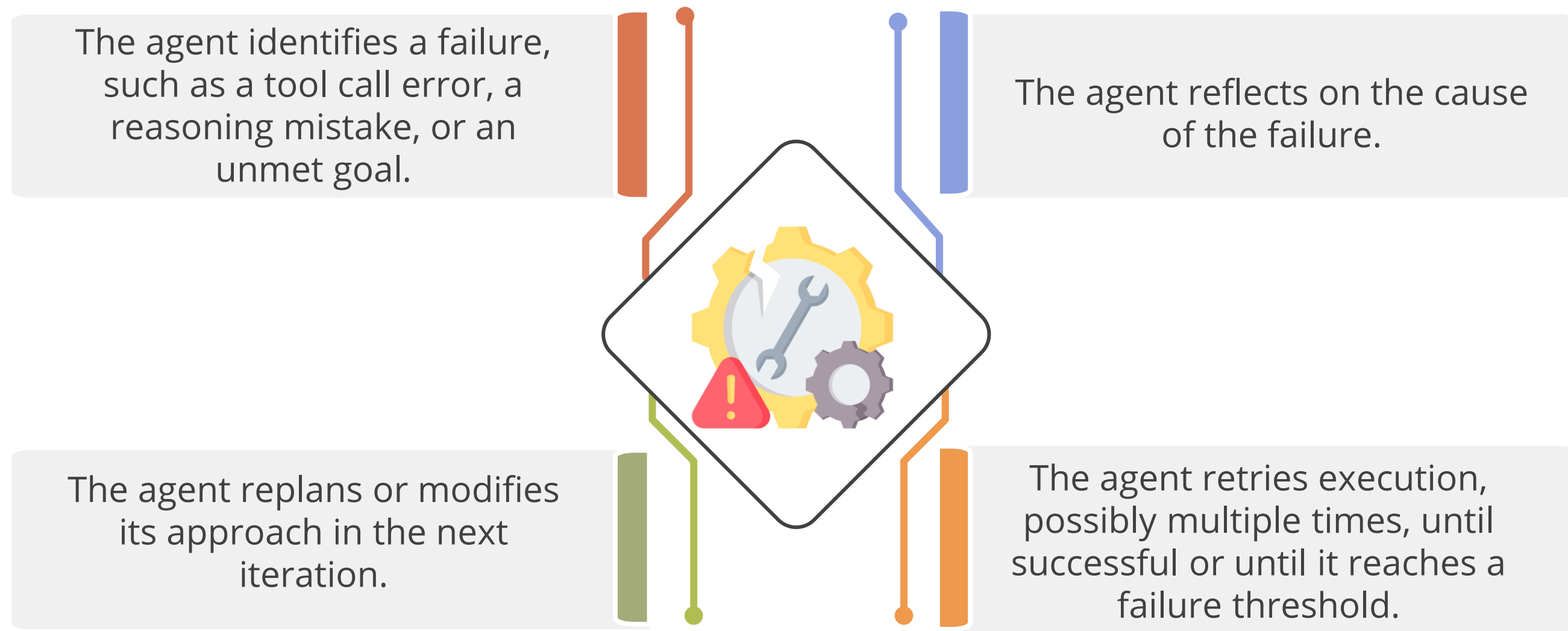
- The reasoning and execution steps are generated together in a single sequence.
- Once reasoning is complete, the system doesn't replan unless restarted.
- It is well-suited for tasks where one-shot reasoning works well.

Plan-and-execute agents

- Tasks are decomposed into subtasks before execution.
- The agent can dynamically replan based on feedback.
- It is useful when external tool calls, multi-agent collaboration, or long-term memory are needed.

Failure Loops in Autonomous Agent Execution

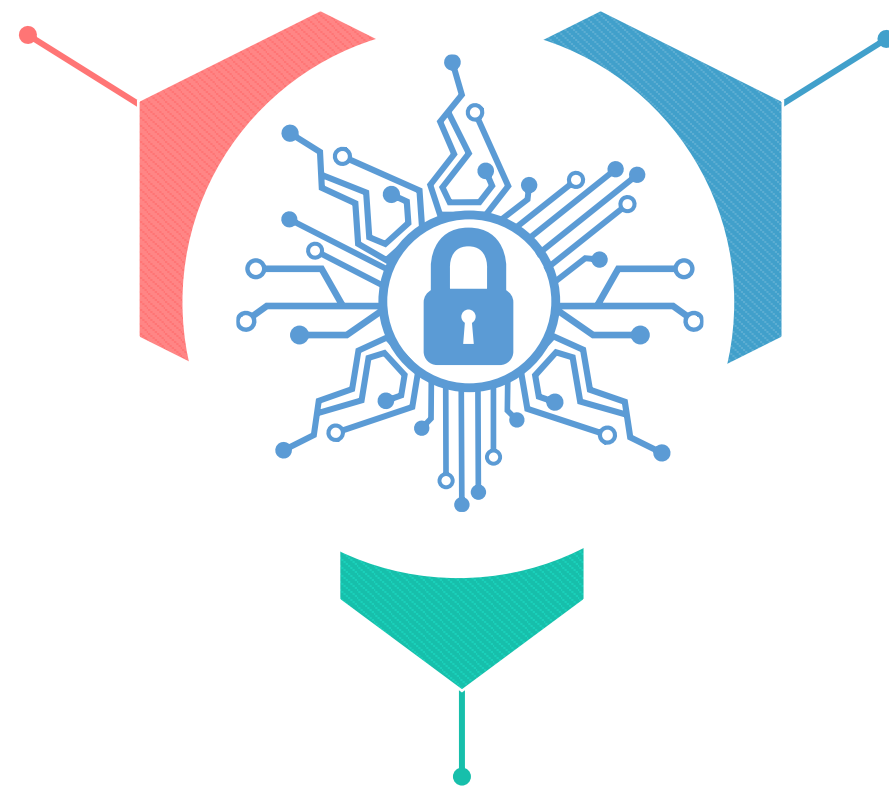
A failure loop occurs when an autonomous agent detects a failure or poor outcome and enters a cycle to recover, replan, and re-execute.



Importance of Failure Loops

Failure loops are important for improving agent reliability and decision-making.

Autonomy: Allows agents to self-correct without human intervention



Resilience: Enables handling of unpredictable environments and tool failures

Optimization: Supports iterative improvements in reasoning and execution

Quick Check



An enterprise AI is coordinating inventory management across multiple warehouses, adjusting orders dynamically based on stock levels and delivery delays.

Which agent type should be chosen and why?

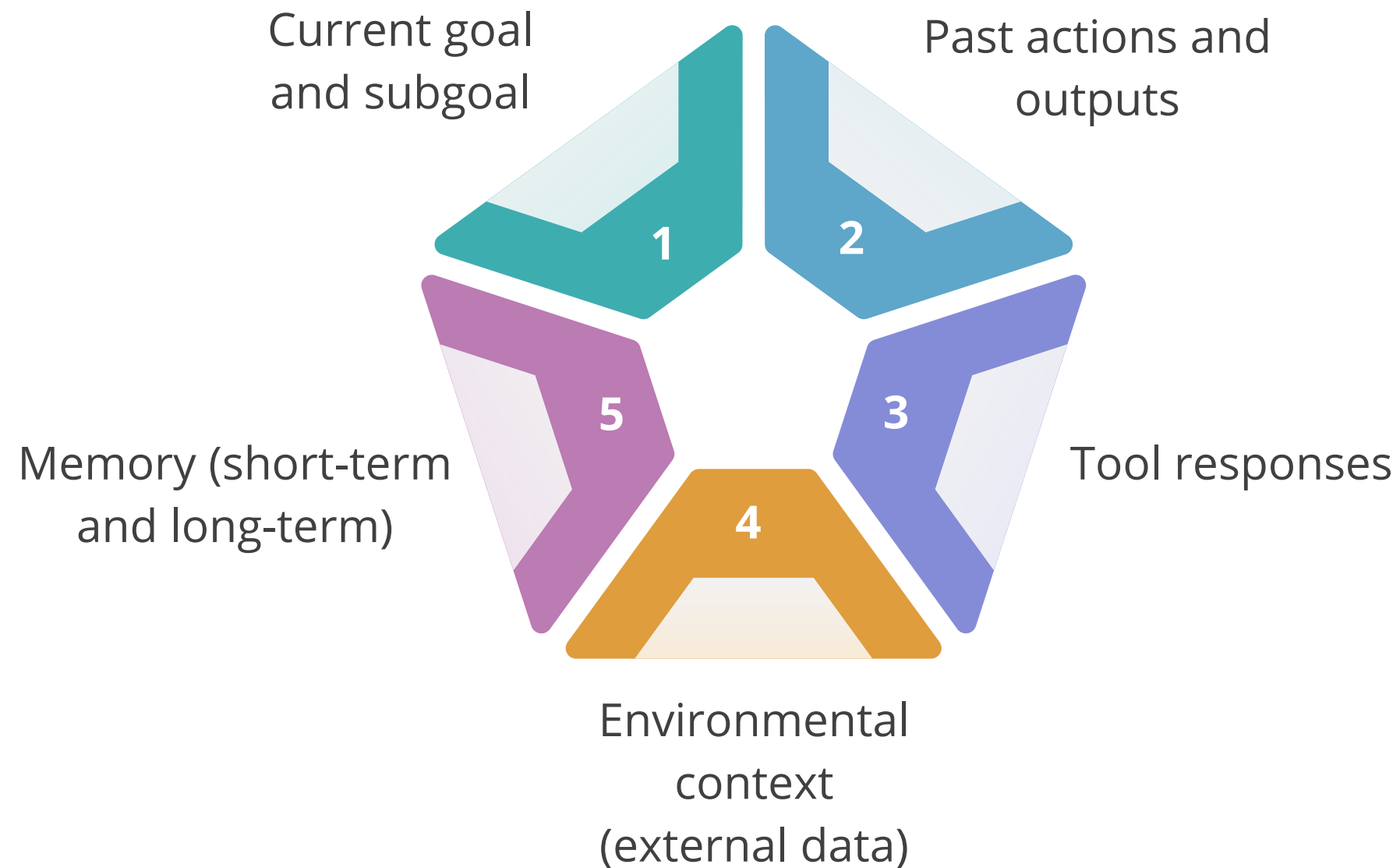
- A. CoT agent, because it is faster.
- B. Plan-and-execute agent, because it supports iterative replanning.
- C. CoT agent, because it does not require memory.
- D. Plan-and-execute agent, because it is faster.



Understanding and Managing Agent State

What Is Agent State?

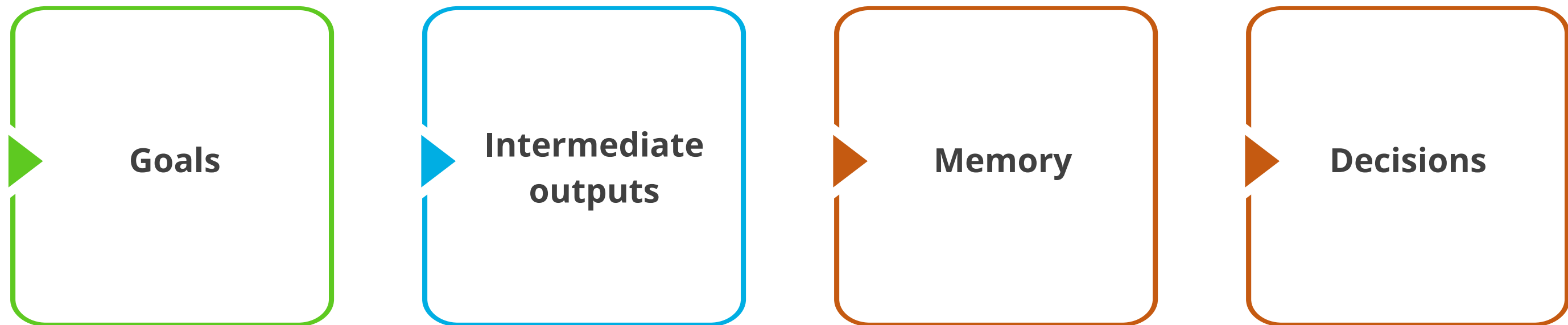
Agent state is a structured record of what the agent knows, has done, and plans to do. It typically includes:



Managing Agent State Across Steps in Agentic AI

Agentic AI systems require agent state management to support reasoning, continuity, and decision-making in complex workflows.

The agent state maintains information on:



Advantages of Maintaining Agent State

Maintaining agent state provides the following benefits:

Context retention

Enables agents to retry failed steps using stored intermediate outputs.

Collaboration

Manages agents so that they share states across multi-agent systems for effective coordination.

Adaptive planning

Updates internal state with new information to support dynamic replanning.

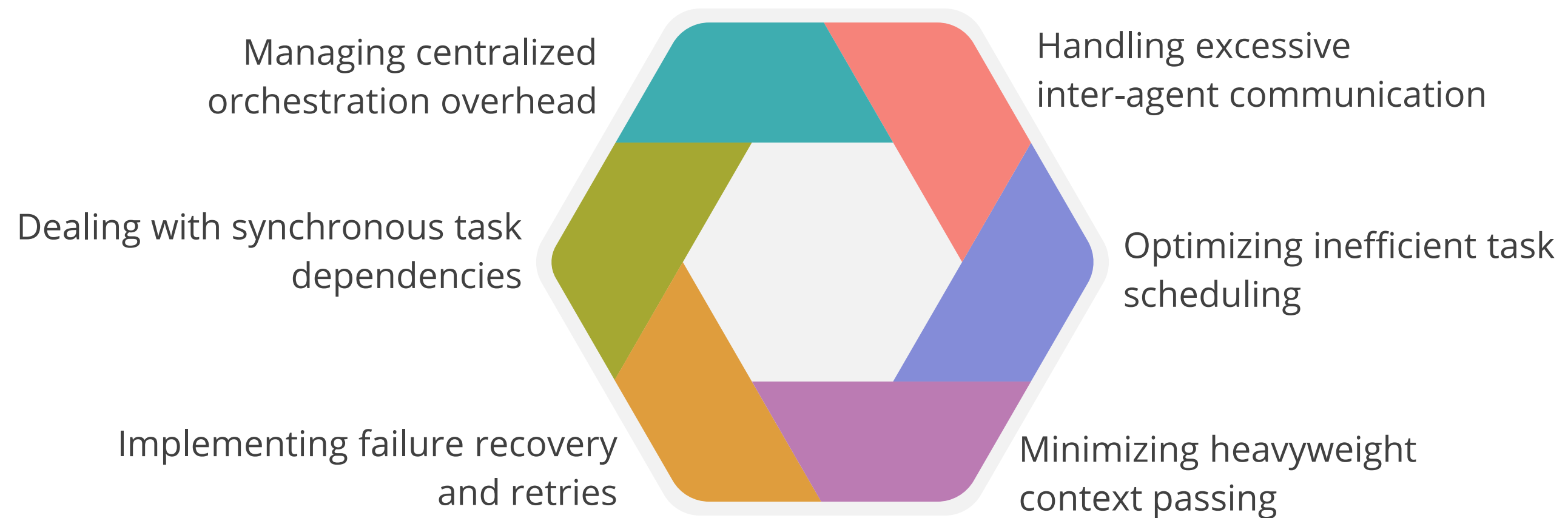
Failure recovery

Supports recovery and resilience by retaining outputs for step-level retries.

However, maintaining agent state and enabling complex coordination can introduce orchestration challenges that impact performance.

Orchestration-Driven Latency Bottlenecks

Agentic orchestration introduces latency due to factors such as coordination overhead, task dependencies, and communication delays. Key characteristics include:



Real-World Example: Sales Assistant Agent

A sales assistant agent helps sales representatives by researching prospects, drafting personalized outreach emails, and updating CRM entries.



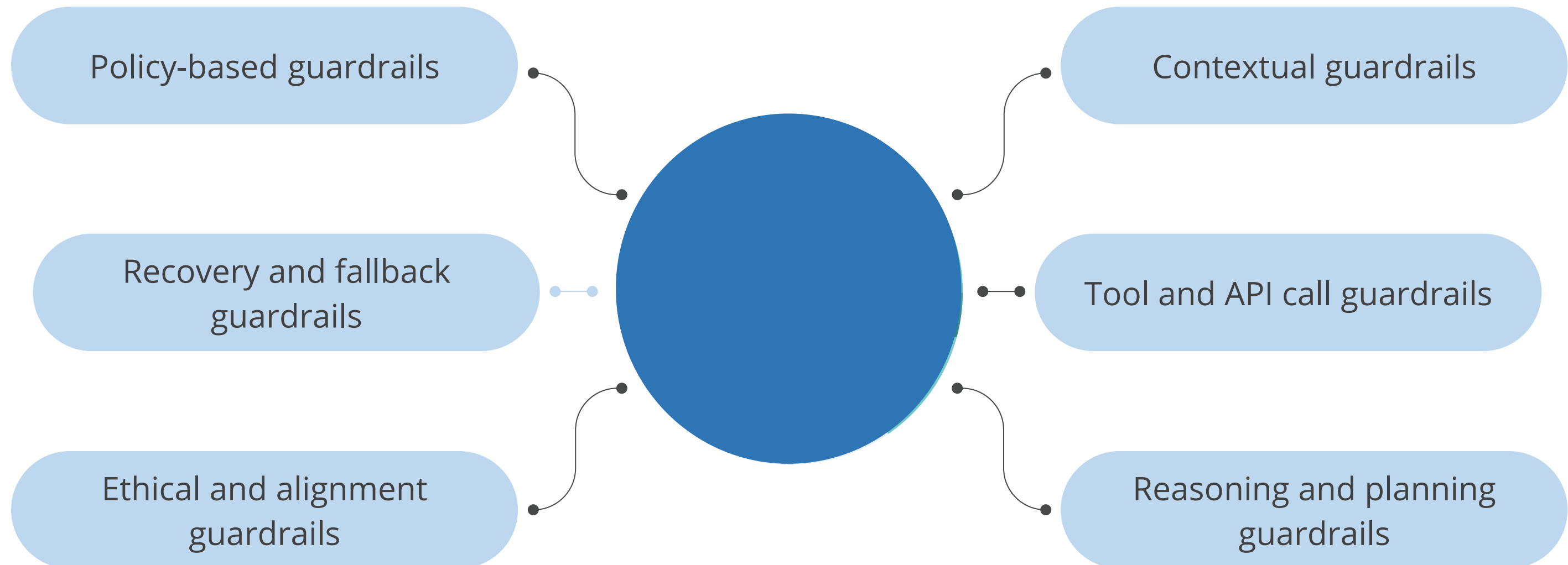
It runs on an orchestration layer that coordinates planning, tool usage, and multi-step execution.



Introduction to Safety, Guardrails, and Monitoring

Guardrails for Agent Behavior Control

Guardrails are mechanisms that constrain and align agent behavior with operational, ethical, and security standards. Key categories include:



LangSmith and Guardrails for Monitoring Agents

Monitoring agents help ensure correctness, reliability, and safety in agentic AI systems.

Challenges

- Generate biased, harmful, or inaccurate responses when handling complex queries
- Miss policy violations due to lack of continuous oversight
- Undermine user trust and expose organizations to brand risks through inconsistent responses

LangSmith and Guardrails for Monitoring Agents

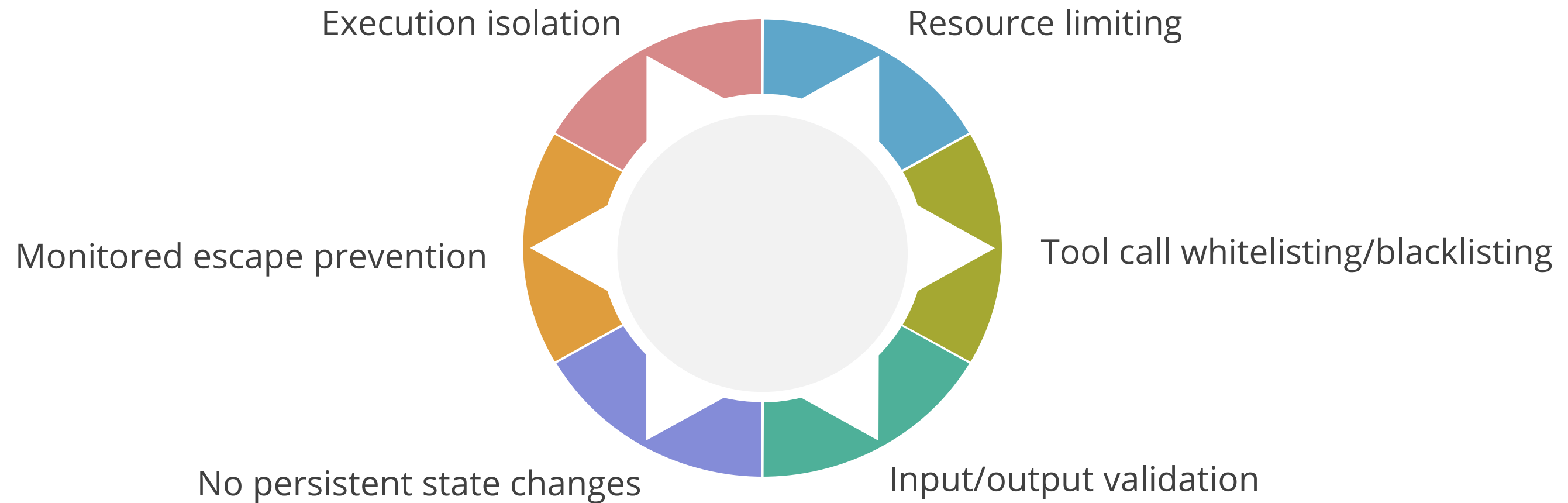
Solution

- Detecting biased, incorrect, or unsafe outputs in real time prevents issues from reaching end-users.
- Addressing issues proactively prevents escalation into costly failures.
- Enforcing clearly defined rules and checkpoints ensures alignment with approved workflows, policies, and tone guidelines.

Monitoring alone isn't enough; securing agents also requires policy enforcement, oversight, or human-in-the-loop control through interrupt, reset, and abort mechanisms.

Sandboxing to Prevent Uncontrolled Actions

Sandboxing isolates agentic AI actions within a controlled environment to reduce the risk of unintended or harmful behavior. Key characteristics include:



Interrupt, Reset, and Abort Strategies

While sandboxing secures the agent's boundaries, policy enforcement, oversight, and human-in-the-loop controls through interrupt, reset, or abort are also essential.

Interrupt

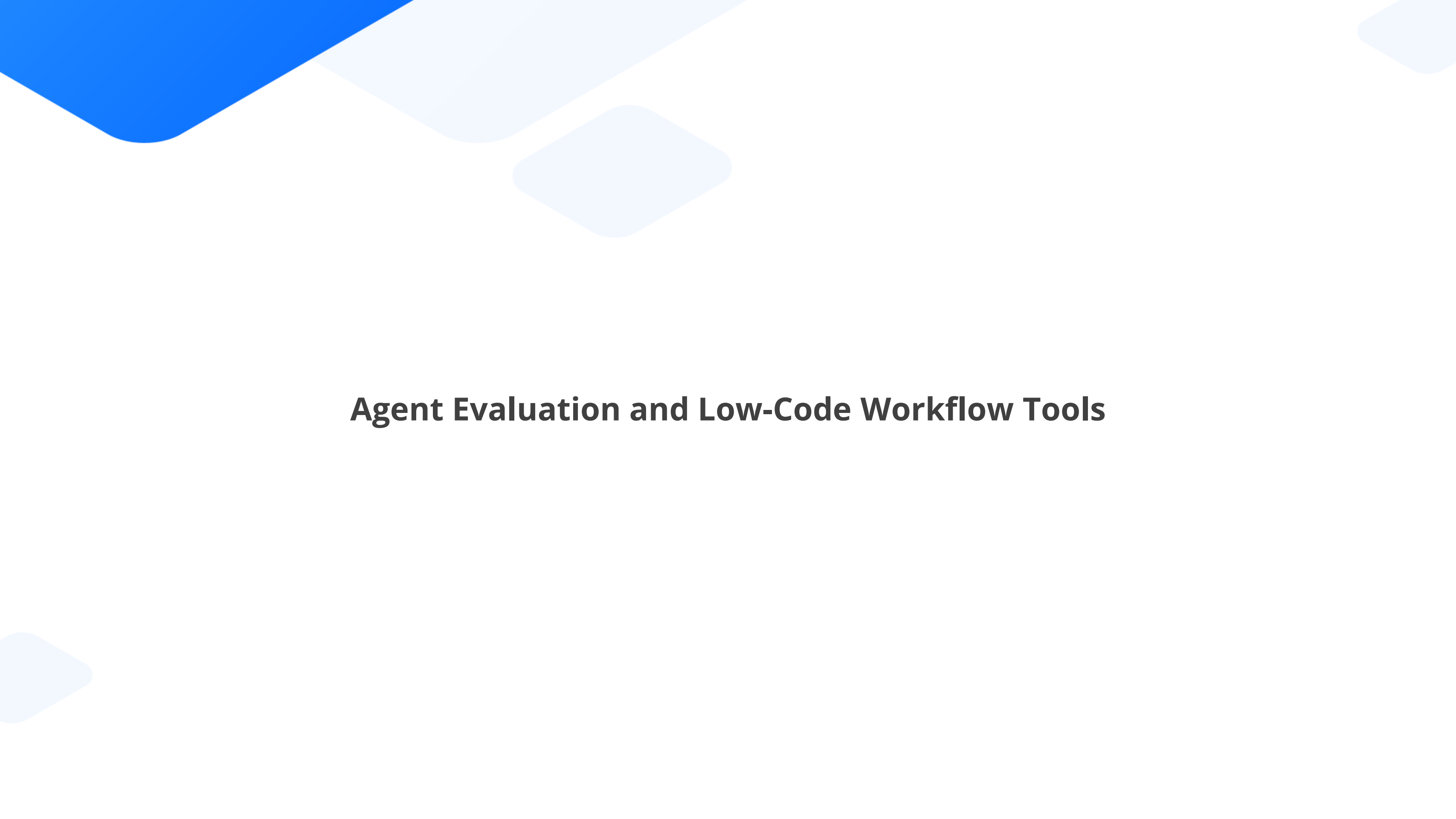
- Temporarily pauses the agent's ongoing task
- Allows human or system intervention to adjust inputs or settings

Reset

- Reverts the agent to a safe default state without shutting down entirely
- Clears temporary context or incorrect reasoning while preserving stable configurations

Abort

- Fully stops the agent's current operation and blocks further actions
- Triggers safety protocols, such as logging, alerts, or failover to human agents



Agent Evaluation and Low-Code Workflow Tools

Agent Evaluation Metrics

Evaluating agentic AI requires technical, reliability, and user-centric metrics to ensure safe, effective, and consistent performance. Key evaluation metrics include:

01 Success rate

02 Tool accuracy

03 Efficiency

04 Completion time (Latency)

No-Code/Low-Code for Agent Prototyping

It allows rapid prototyping of agentic AI workflows without extensive programming.

Faster iteration

Build, test, and modify agent workflows quickly

Integration ready

Connect to APIs, databases, and SaaS tools easily

Cost-effective

Avoid heavy engineering resources during early experimentation

Collaborative

PMs and tech teams can visually co-design workflows.

Workflow Automation Tools: Zapier

Zapier is designed for business users who need quick, no-code automation.

Strengths



- Provides a user-friendly interface for simple trigger-to-action workflows
- Supports a large library of application integrations with over six thousand apps
- Caters to non-technical users through intuitive design

Limitations



- Lacks conditional logic and branching support
- Does not have advanced debugging and custom scripting capabilities
- Incurs high costs due to pay-per-task pricing at scale

Workflow Automation Tools: Make.com (formerly Integromat)

Make.com supports complex automations with rich logic and advanced data transformations

Strengths



- Builds visual scenarios with rich logic, loops, and conditional paths
- Performs multi-step data manipulation effectively
- Supports cost-effective execution for high-volume operations

Limitations



- Has a steeper learning curve
- Has limited local or self-hosted options due to cloud-first design

Workflow Automation Tools: n8n

It is designed for developers or teams needing open-source and self-hosted automation.

Strengths



- Supports self-hosted and privacy-friendly deployments, ideal for sensitive data workflows
- Enables high customization through JavaScript function nodes
- Offers free local deployments and enterprise-ready capabilities

Limitations



- Requires technical knowledge for setup and maintenance
- Offers fewer out-of-the-box app integrations than Zapier

Connecting Agents to Real-World APIs

Agentic AI systems autonomously retrieve data, take action, and integrate with real-world APIs.

Step	Agent action	Outcome for PM
Data retrieval	Fetch open tickets, completed tasks, and blockers.	List of current sprint tasks with statuses
Team communication analysis	Scan slack for key project discussions, blockers, and approvals	Extracted conversations tagged as critical updates
Document review	Retrieve the latest design docs or meeting notes	Pulls updated files and highlights changes
Summarization	Summarize tasks completed, ongoing issues, and decisions made	A concise project update with key points

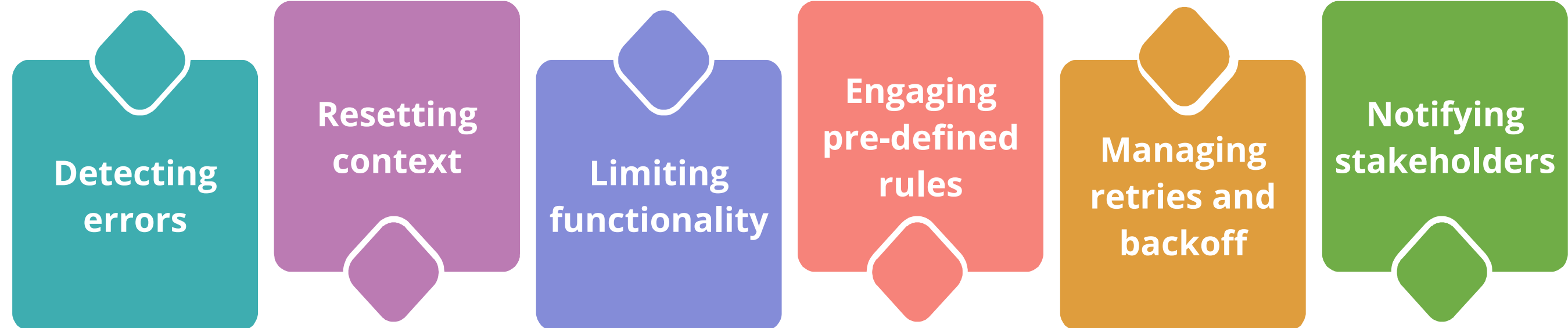
Structured vs. Dynamic Prompt Routing

Prompt routing determines how agentic AI selects the best path to process a request, balancing control and adaptability. The comparison between structured and dynamic prompt routing is given below:

Aspect	Structured prompt routing	Dynamic prompt routing
Definition	Fixed, rule-based routing following predefined workflows	Context-aware, real-time routing determined by models or heuristics
Adaptability	Requires manual updates for new scenarios	Automatically adjusts based on input and context.
Predictability	Very high consistency and repeatable outcomes	Outcomes may vary due to dynamic decision-making.
Debugging and monitoring	Easy and clear rules and traceable steps	Reasoning paths can change between runs.

Error Handling and Recovery in Agent Execution

Error handling and recovery options help agents remain reliable, respond gracefully to failures, and maintain user trust. It helps agents in:



Transparency and Explainability in Tool Usage

Transparent tool usage builds trust by showing how and why an agent chooses specific tools or services during task execution. This approach is characterized by the following attributes:



As trust and transparency grow, product managers must also make strategic choices about orchestration to balance performance, cost, and reliability.

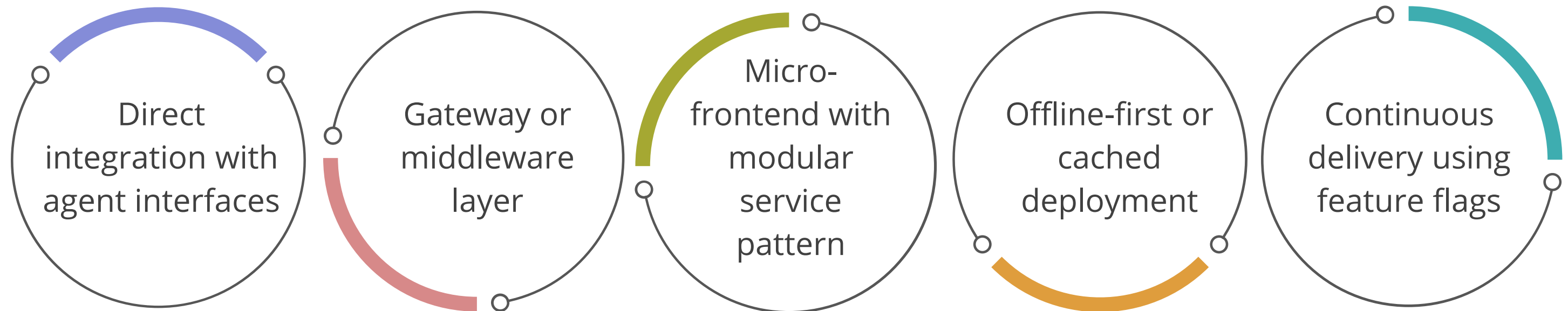
Strategic PM Lens on Orchestration Choices

Product managers should view orchestration not just as a technical decision but as a strategic lever to balance speed, accuracy, cost, and user trust. PMs focus on the following outcomes:



Deployment Patterns for Agentic Products

The application layer serves as the user-facing gateway in agentic products. Deployment patterns must balance speed, reliability, scalability, and continuous improvement.



Quick Check

A product manager (PM) needs an up-to-date list of tasks and blockers for the current sprint. The agent fetches open tickets and completed tasks automatically.

Which step from the table (slide 51: Connecting Agents to Real-World APIs) describes this action?

- A. Document review
- B. Data retrieval
- C. Summarization
- D. Team communication analysis





Exploring the Application Layer

Application Layer

It transforms abstract agentic AI capabilities into practical, user-friendly solutions.



Application Layer – The User Experience Face of AI Agents

The application layer is the main interface where users interact with agentic AI systems. It plays a key role in usability and shapes how intelligent and trustworthy the system feels.

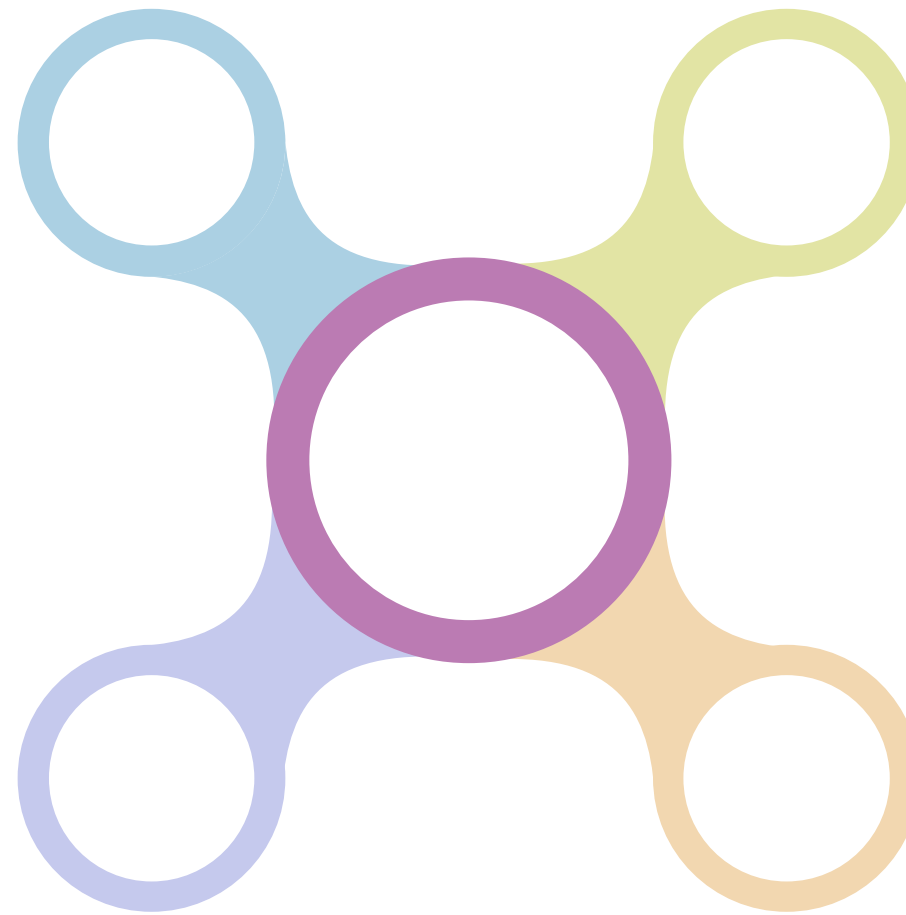
Its key characteristics are:

User-friendly
insights

Simplified
outputs

Multi-channel
interaction

Trust and
engagement:



Maintaining Transparency and Trust in AI Agents

A transparent and explainable user interface builds trust by helping users understand how and why an AI agent makes decisions.

Displays reasoning steps or summaries to clarify decision-making processes

Provides confidence scores or uncertainty indicators to manage user expectations

Offers clear action history that shows what the agent did and why

Allows users to ask “why” or “how” questions to increase control and trust

Uses simple visualizations to explain complex AI logic

Beyond transparency, trust also depends on predictable system behavior and user-friendly retry options when things go wrong.

Predictability and Retry Patterns in UX

Predictable interaction flows and clear retry mechanisms reassure users when agentic AI systems fail or delay responses. Its importance can be inferred from the points below:



Provides consistent response patterns so users know what to expect in similar situations



Communicates delays or failures proactively with clear status updates



Offers simple retry options to reduce frustration during temporary errors



Uses visual cues such as progress indicators or step confirmations to maintain user confidence



Ensures retry attempts do not repeat errors by incorporating adaptive feedback

Designing UX with Chat, Form-Fill, and Hybrid Modes

Agentic AI systems use different interaction modes to balance natural conversation, structured input, and task efficiency. The following are the interaction modes:

- **Chat mode:** Supports natural, conversational interactions; best for exploratory or open-ended tasks
- **Form-fill mode:** Provides structured fields for fast, precise data entry; ideal for transactional or compliance-heavy workflows
- **Hybrid mode:** Combines conversation with structured inputs, allowing guided dialogs for complex tasks

Staged Information Delivery in AI Agents

It presents agent-generated information in manageable layers, reducing cognitive overload and improving decision-making.

Reveals only essential information at first, with options to explore details gradually

Supports decision confidence by providing more depth only when requested

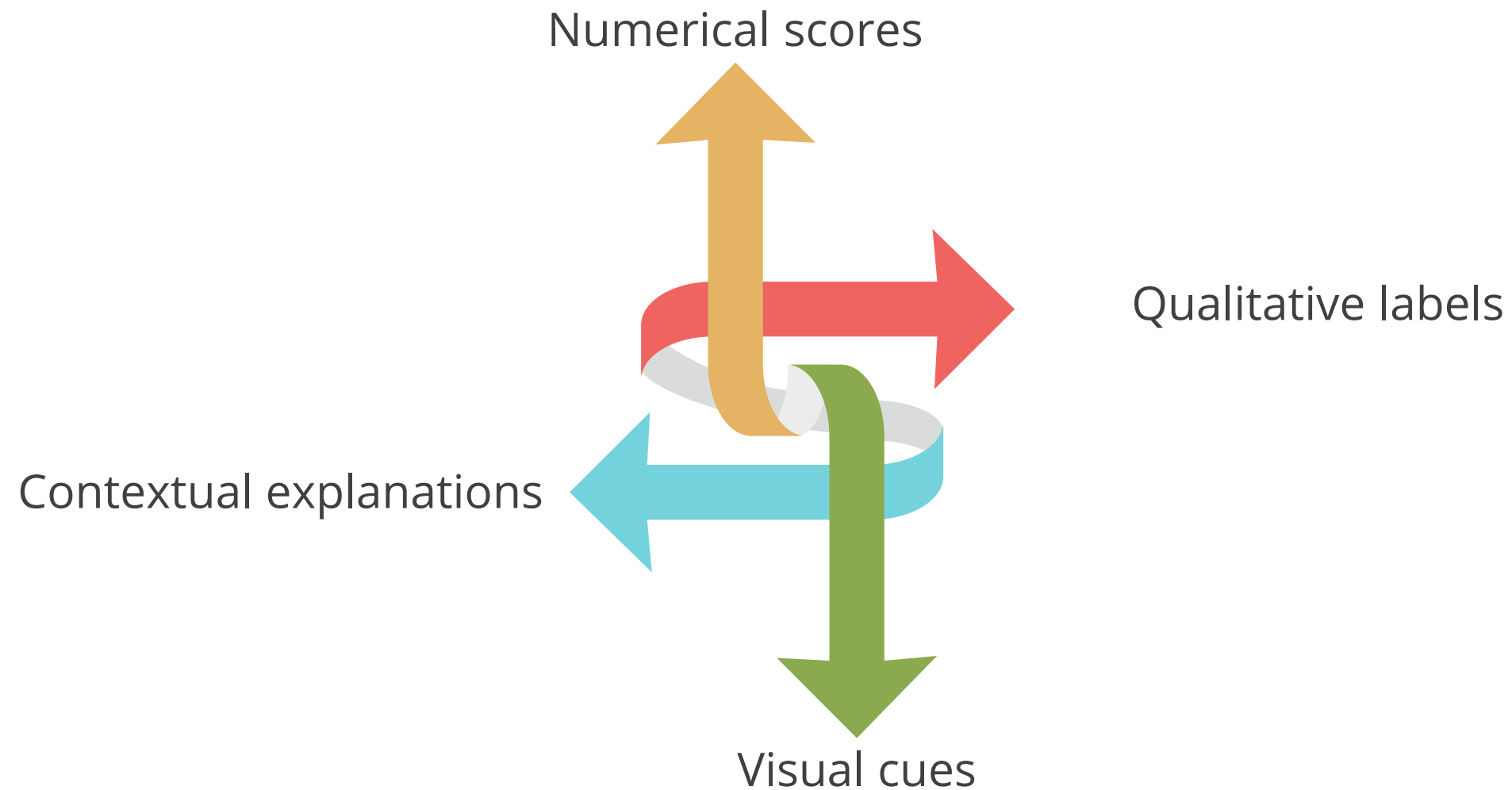
Balances transparency with simplicity to avoid overwhelming non-technical users

Organizes outputs into logical tiers, including summary, key factors, and full reasoning trace

Uses clear visual hierarchy and interactive elements to guide exploration

Confidence Indicators in Agent Responses

Displaying confidence indicators at the application layer helps users assess the reliability of an agent's response, improving trust and decision-making. The following are the types of confidence indicators:



To complement these indicators, agents must also manage ambiguity in their responses to ensure clarity and guide users effectively.

Handling Ambiguity in Natural Language Output

Agents must manage ambiguity in their responses to maintain clarity, build trust, and guide users toward better outcomes. Ambiguity can be handled by:

Clarification Questions

Example: *Do you mean product refund or warranty replacement?*

Disambiguation Options

Example: *I found 3 accounts under your name. Which one should I update?*

Confidence-Based Assumptions (with Opt-Out)

Example: *Assuming you want to book the earliest available flight. Is that correct?*

Explicit Uncertainty Communication

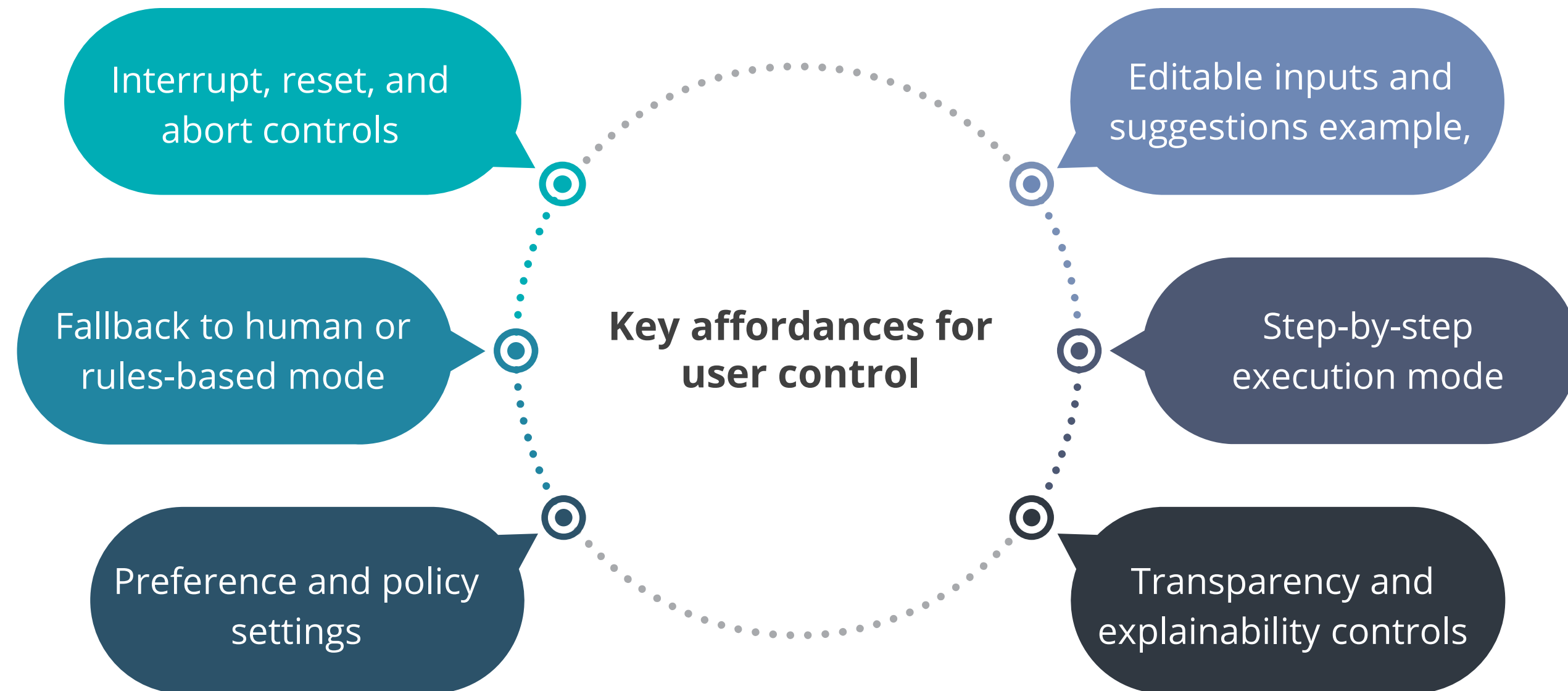
Example: *I am not completely sure about this medical dosage. Would you like me to check with a specialist?*

Progressive Disclosure

Example: *Give me more details on the above given topic*

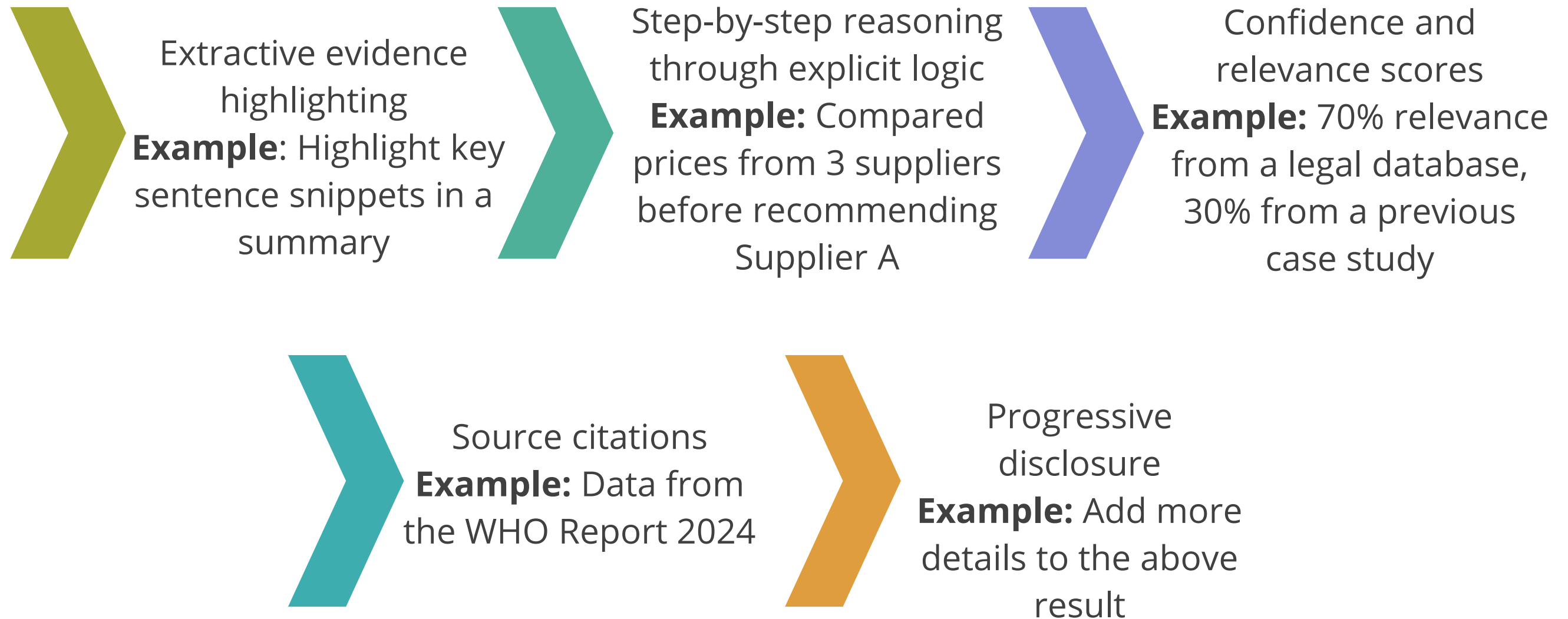
AI Affordances for User Control

Affordances in AI interfaces give users clear ways to guide, adjust, or override agent behavior, increasing trust and usability.



Agent Output Grounding

Grounding agent outputs means presenting evidence, sources, or reasoning steps behind an answer to improve reliability and build trust. Key grounding techniques include:



Example: Research Assistant Interface

A research assistant agent helps academic researchers summarize papers, extract key findings, and recommend related studies. The following illustrates the user experience design for such an interface.

Context Memory

Active papers

- Greenhouse Gas Emissions and Land Use Change from Crepland intensification: A Spatially-Explicit Life C...
- Natural climate solutions

Filters

- Pinned findings

NCS aim to reduce greenhouse gas emissssions and increase carbon sequestration

What are natural climate solutions?

09:41

Natural climate solutions (NCS) are actions to protect, manage, or restore natural and modified ecosystems to address climate change.

Show work

09:41

MEDIUM

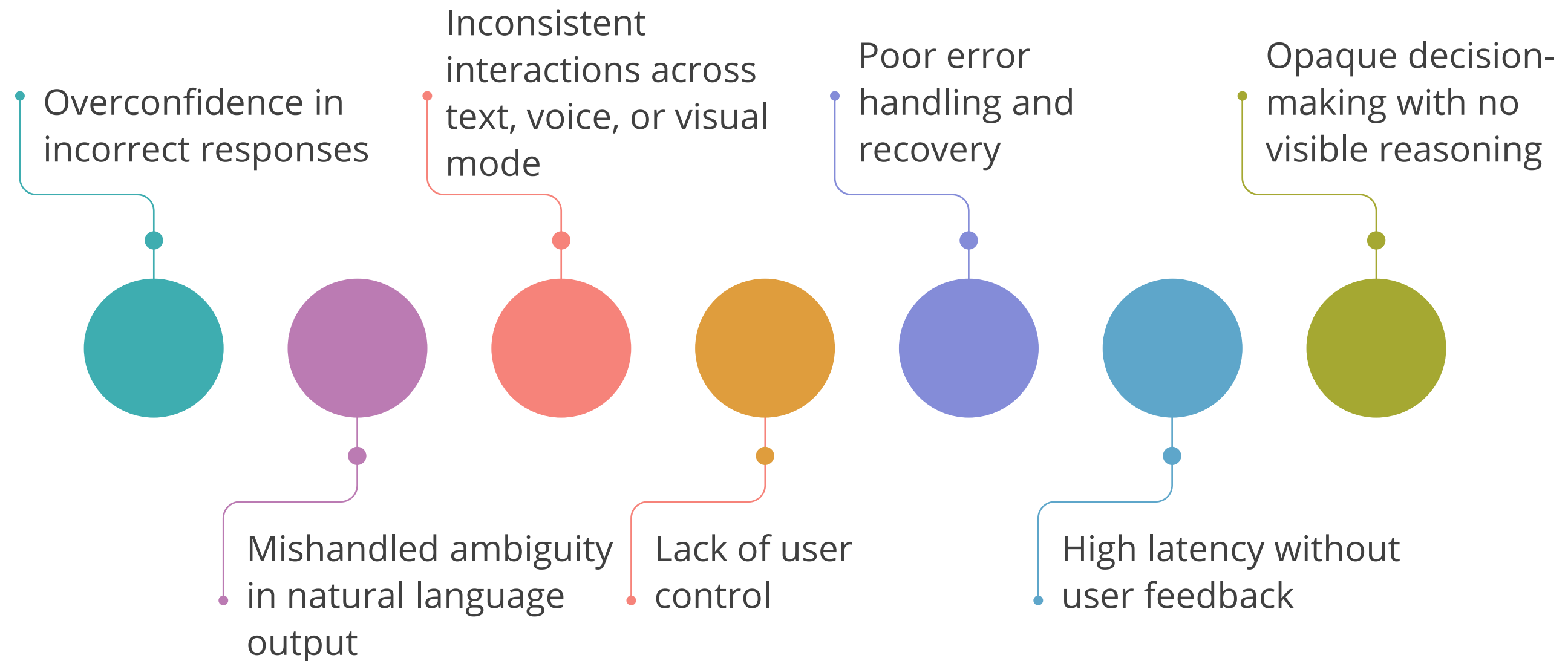
Explain

Retry

Message...

UX Failure Cases in Agentic Products

Poor UX in agentic AI products often stems from mismatched user expectations, lack of transparency, and weak error handling.



Bridging UX Challenges with the Right Prototyping Tools

To address the UX pitfalls in agentic AI products, such as inconsistent interactions, poor error recovery, and a lack of transparency, teams need rapid, iterative prototyping.



The right tools can help validate design decisions early, test interaction models, and ensure that user expectations align with system behavior.

Prototyping Tools: Bubble, Retool, and Streamlit

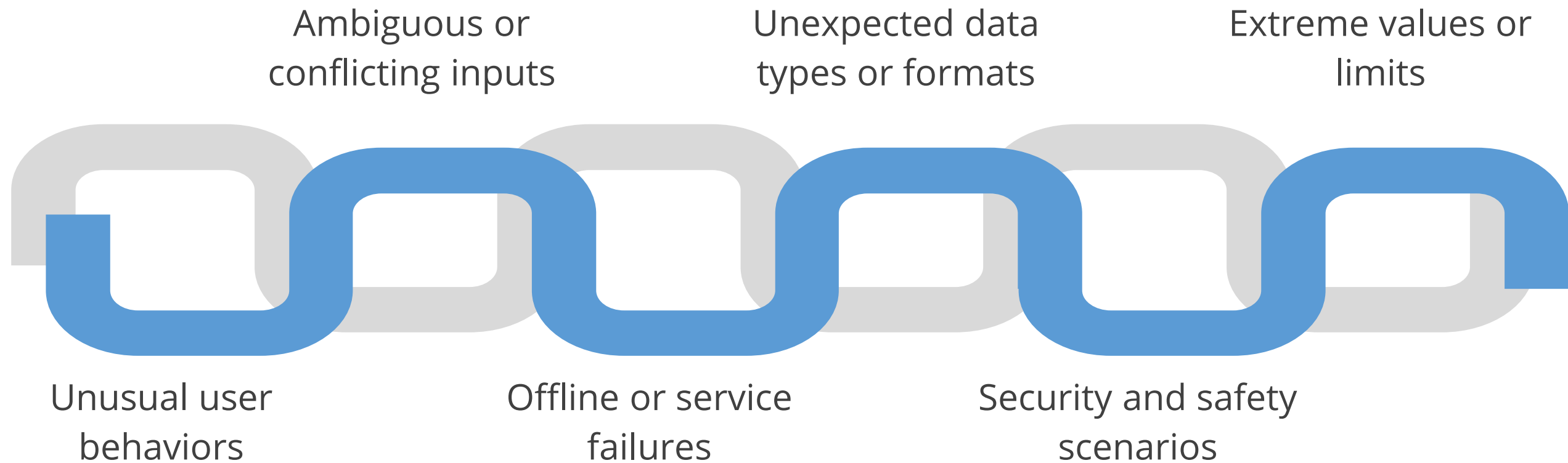
These tools support rapid prototyping and deployment of agentic AI interfaces. They offer a balance between speed, customization, and technical complexity.

Aspect	Bubble	Retool	Streamlit
Primary use	No-code web app builder for end-user apps	Internal tool builder for dashboards and workflows	Python-based app framework for data science and ML demos
Ease of use	Very high drag-and-drop, no coding required	Moderate low-code, visual editor with SQL/API connectors	Requires Python knowledge
Best for	Non-technical founders or PMs building MVPs	Ops teams, analysts, and engineers building internal tools	Data scientists, researchers, and ML engineers
Integration capability	Basic APIs and plugins	Strong API and database integrations	Full Python ecosystem, AI/ML libraries
Hosting	Cloud-based, managed by Bubble	Cloud or self-hosted	Python app deployable on Streamlit Cloud or custom servers

Interface Testing for Edge Cases

Edge case testing in agentic AI involves ensuring that the system can respond appropriately to rare, unexpected, or extreme inputs without failing, misleading users, or violating safety and usability expectations.

The following are the common edge cases:



Metrics for Evaluating Agent UX Quality

Measuring UX quality in agentic AI products requires a mix of performance, trust, and interaction metrics to ensure a reliable and satisfying user experience.



Performance
and
efficiency

Trust and
transparency

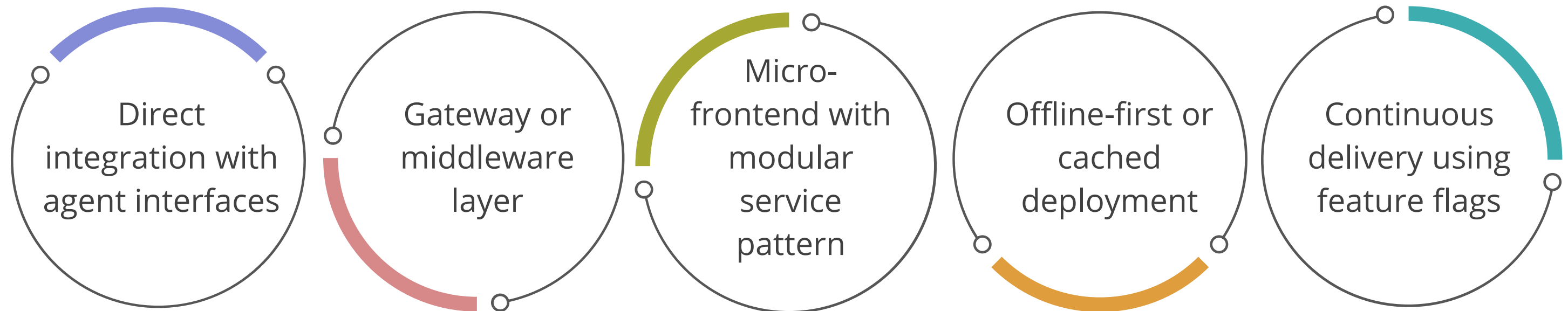
User control
and
autonomy

User
satisfaction
and adoption

Error and
ambiguity
handling

Deployment Patterns for Agentic Products

The application layer serves as the user-facing gateway in agentic products. Deployment patterns must balance speed, reliability, scalability, and continuous improvement.



Key Takeaways

- Agentic frameworks support autonomous agent development by offering tools for planning, reasoning, and multi-step execution.
- User control improves trust and usability by enabling individuals to guide, adjust, or override agent actions.
- Predictable interaction flows foster trust by producing consistent response patterns across similar situations.
- Managing agent state enhances reliability and efficiency by tracking and referencing past decisions.
- Prototyping tools accelerate interface development by balancing speed, customization, and technical demands.
- The application layer serves as the primary interface for agentic AI, shaping both usability and perceived intelligence.



Q&A

